SlateDB

An embedded database built on object store.

How do you build a performant, embedded key-value store on object store?

About Me

Vignesh Chandramohan

- Data Infrastructure @ DoorDash
- Previously: Microsoft Azure, Stream Processing.
- Open Source: SlateDB.

- https://www.linkedin.com/in/vigneshcdata/
- https://datapapers.substack.com/
- @vigneshc.bsky.social

Plan for today

- Motivation.
- What is SlateDB?
- Internals: How is it built?

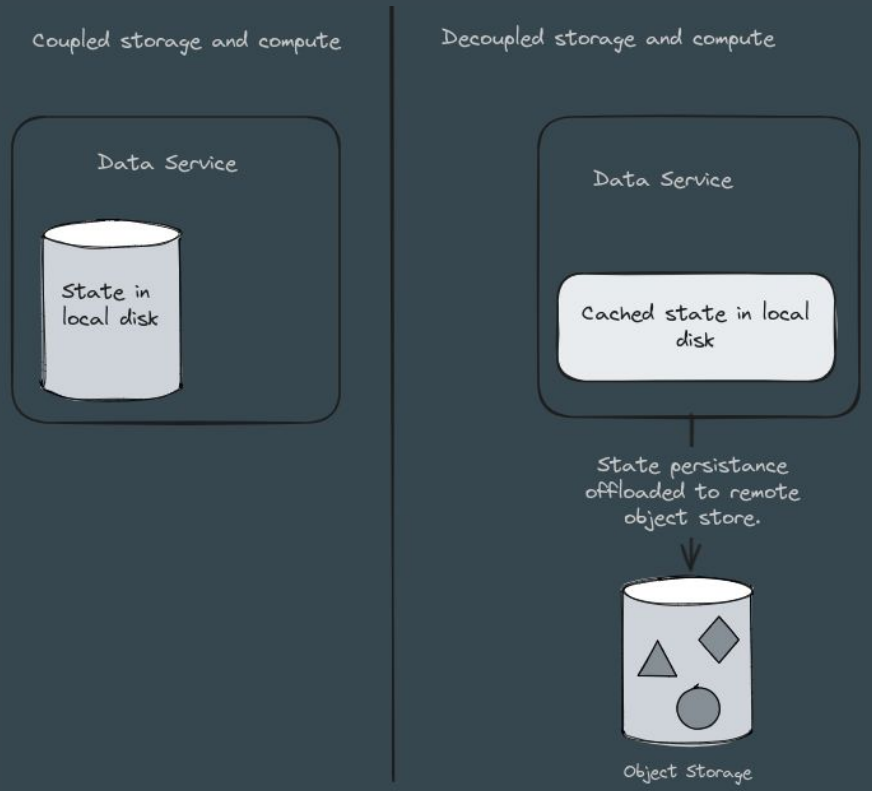
Motivation

The rise of disaggregated storage systems

- Object stores (S3, GCP, Azure Blob Stores) offer attractive properties!
- Architecture that offloads state to S3 is on the rise.

<u>Examples</u>

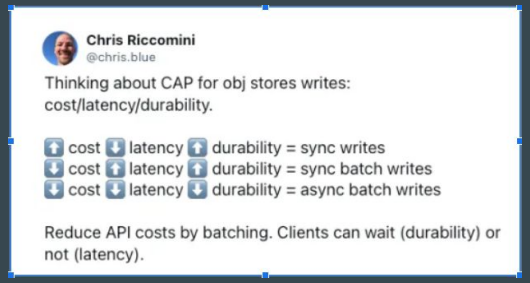
- WarpStream: Kafka compatible streaming platform.
- Iceberg/Delta/Hudi: High performance ACID table storage over cloud object storage.



Object Store based design - 3 common techniques

- Immutable files: New data creates new files. Makes caching easier.

- Batched writes
  - S3 cost for 100k PUT calls: $0.50
  - Increase batching and latency to reduce cost.



- Conditional writes:
  - Write-If-Not-Exists
  - Predictable, numbered file names, causing conflict on overwrites.
    - 00000000001.json

What is SlateDB?

A cloud native embedded storage engine, built on object store.

- In-process (Rust) library.
- Key-value interface.
- All writes go to object store.
- Implemented as a log-structured merge tree (LSM).

SlateDB is designed for use cases that can tolerate **50-100** ms write latency or trade-off additional api cost.

- Stream processing state store.
- Durable execution.
- Durable caches.

SlateDB

Usage

Responsive :

https://www.responsive.dev/ - Hosted Kafka streams. Uses SlateDB for a custom state store.

Tensorlake :

https://www.tensorlake.ai/ - Seamlessly ingest and convert docs into structured context for LLM. Compute engine uses SlateDB to store input / output data from customer functions.

Embucket :

https://github.com/embucket/embucket - Snowflake compatible lakehouse platform, uses SlateDB for metadata persistence.

```rust
// create object store. Cloud object store would be initialized here.
let object_store = Arc::new(InMemory::new());

// open the DB. This creates an empty wal and manifest.
let kv_store = Db::open_with_opts(path, db_options.clone(), object_store.clone()).await?;

// Simple put. This will create a wal file: 0002.sst
let k1 = b"key1";
let v1 = b"value1";
kv_store.put(k1, v1).await?;
```

1

```rust
// set durable = false. Write a large batch.
let write_options = slatedb::config::WriteOptions {
    await_durable: false,
};

for i in 0..500 {
    let k = format!("key{}", i);
    let v = format!("value{}", i);
    kv_store
        .put_with_options(
            k.as_bytes(),
            v.as_bytes(),
            &slatedb::config::PutOptions::default(),
            &write_options,
        )
        .await?
}
```

2

```rust
// explicitly trigger a flush. This writes the batch to object store.
kv_store.flush().await?;
```

3

SlateDB Basic Usage.

https://crates.io/crates/slatedb

https://tinyurl.com/slatedb-demo

SlateDB Internals - the plan

1. Writes

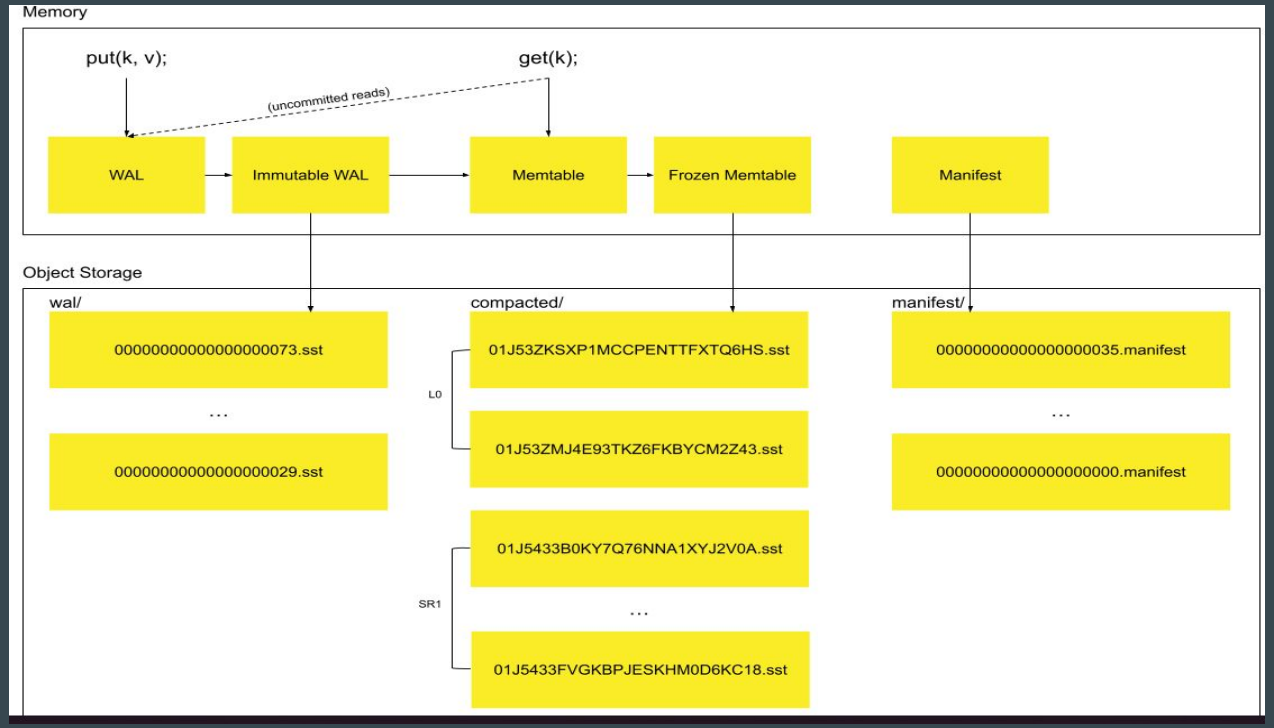
2. Compaction

3. Reads

4. Manifest

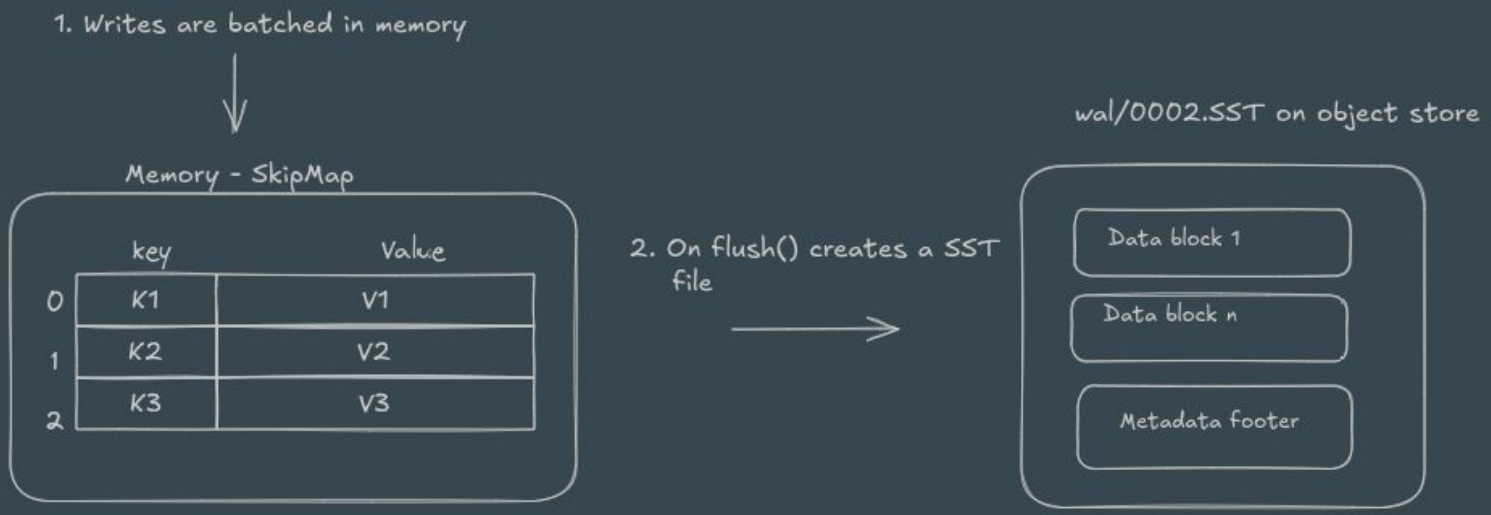
5. Checkpoints

SlateDB - Object store & memory state



- Represents state of a slatedb instance.
- We will walk through these in steps.

Writes (Step 1) : Write WAL SST every flush_ms or on flush()

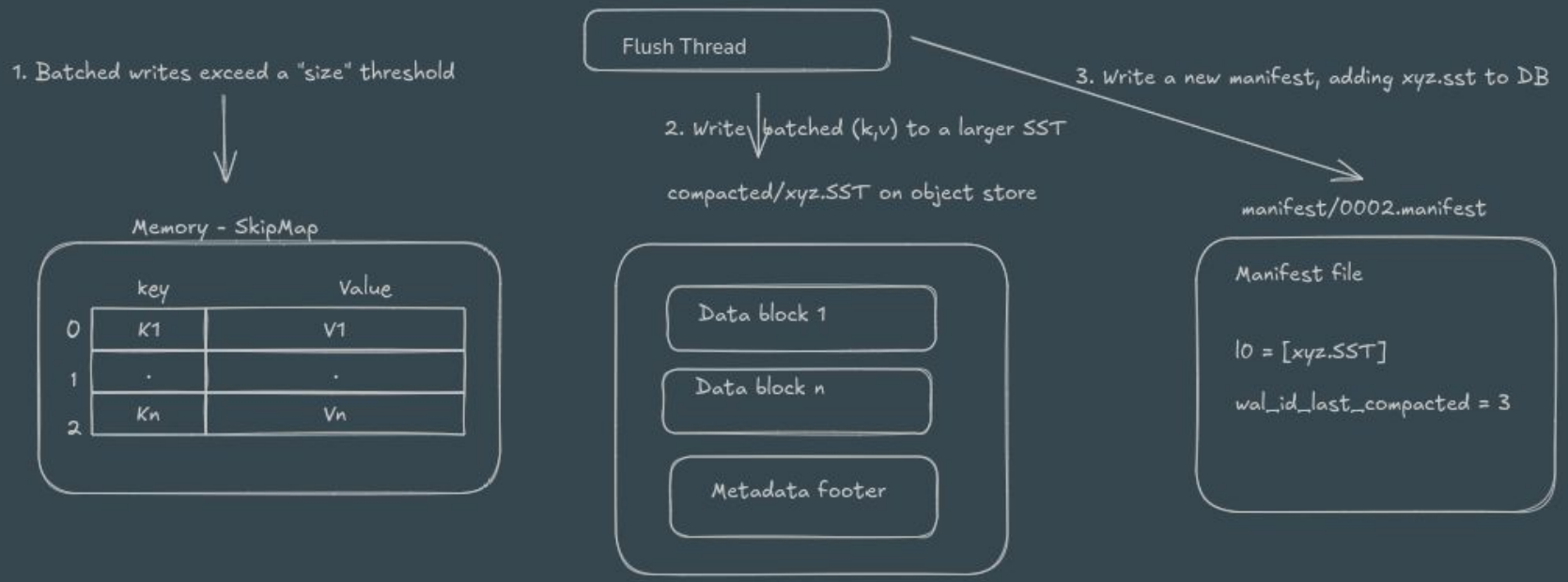


Terminology:

[SST file]: File with sorted key-value pairs, and metadata such as bloom filter and offset of keys .

[WAL]: SST files in "WAL" folder. These are SST files written every flush_ms or when flush() is called.

Writes (Step 2): Write a larger L0 SST file every l0_sst_size_bytes



Terminology:

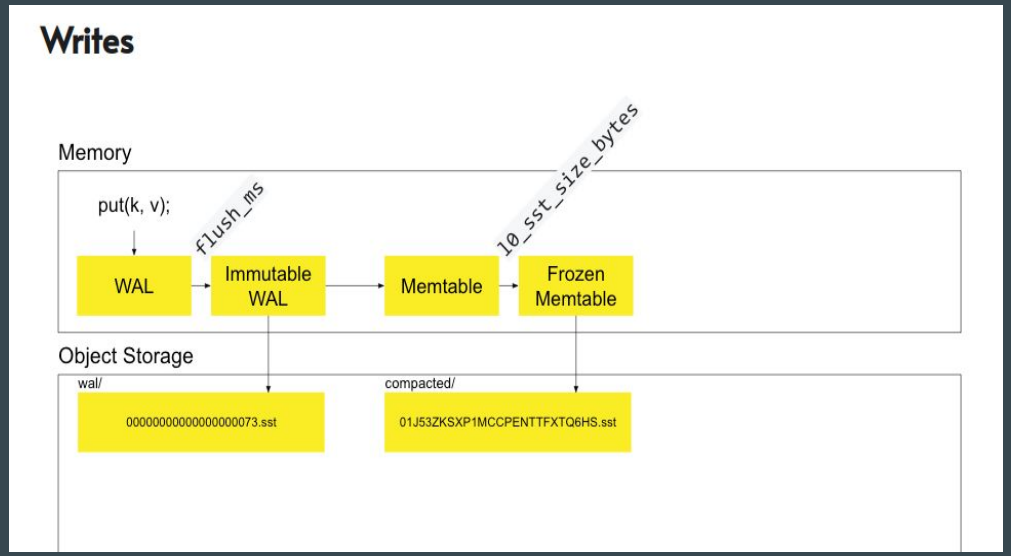
[Manifest file]:
* Describes the state of the DB at a point in time.
* i.e. Contains the list of SST files that are part of the DB.

Writes - step by step

	In-memory wal	Memtable	ObjectStore
1. Write (K1, V1).await	empty	(k1, v1)	wal/0001.SST manifests/0001.manifest
2. Write (K2, V2) await_durable = false	(k2, v2)	(k1, v1)	wal/0001.SST manifests/0001.manifest
3. after flush_ms or flush()	empty	(k1,v1) (k2,v2)	wal/0001.SST wal/0002.SST manifests/0001.manifest
4. Write l0_sst_size_bytes amount of data	empty	empty	wal/0001.SST wal/0002.SST compacted/xyz.SST manifests/0001.manifest manifests/0002.manifest

```
{
  // 0002.manifest
  "manifest_id": 2,
  "writer_epoch": 1,
  "wal_id_last_compacted": 2,
  "l0": ["xyz.SST"]
}
```

*SlateDB writes an empty WAL on writer initialization. It is ignored in this picture.

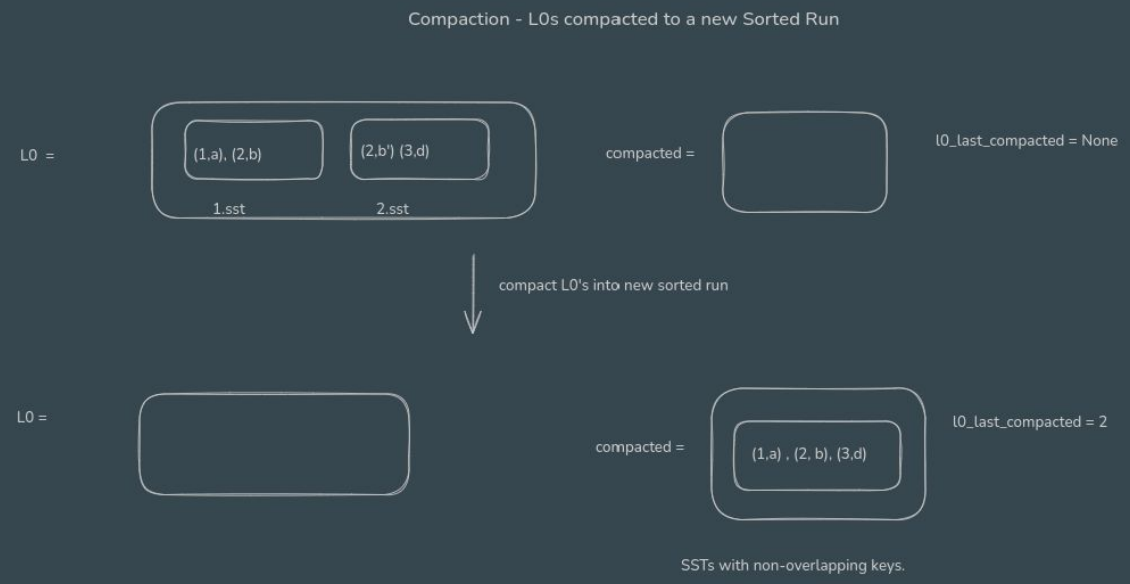



```
some-bucket/
├─ manifest/
│  ├─ 00000000000000000000.manifest
│  ├─ 00000000000000000001.manifest
│  ├─ 00000000000000000002.manifest
├─ wal/
│  ├─ 00000000000000000000.sst
│  ├─ 00000000000000000001.sst
│  ├─ ...
├─ compacted/
│  ├─ ...
```

- Writes are durable once they are written to wal/
- WAL is written on flush() [or] after flush_ms
- WAL is numbered incrementally, and are written with if-not-exists.

Compaction

- Takes a set of SST files, and produces non-overlapping sorted runs.
- Each compaction creates a new manifest file.



Compaction - more examples

1. Initial state

L0 = [1, 2]
Sorted Run=[S1]

2. Compact [1,2] and create new Sorted Run

L0 = []
Sorted Run = [S2, S1]
S2 = contains contents from [1,2]

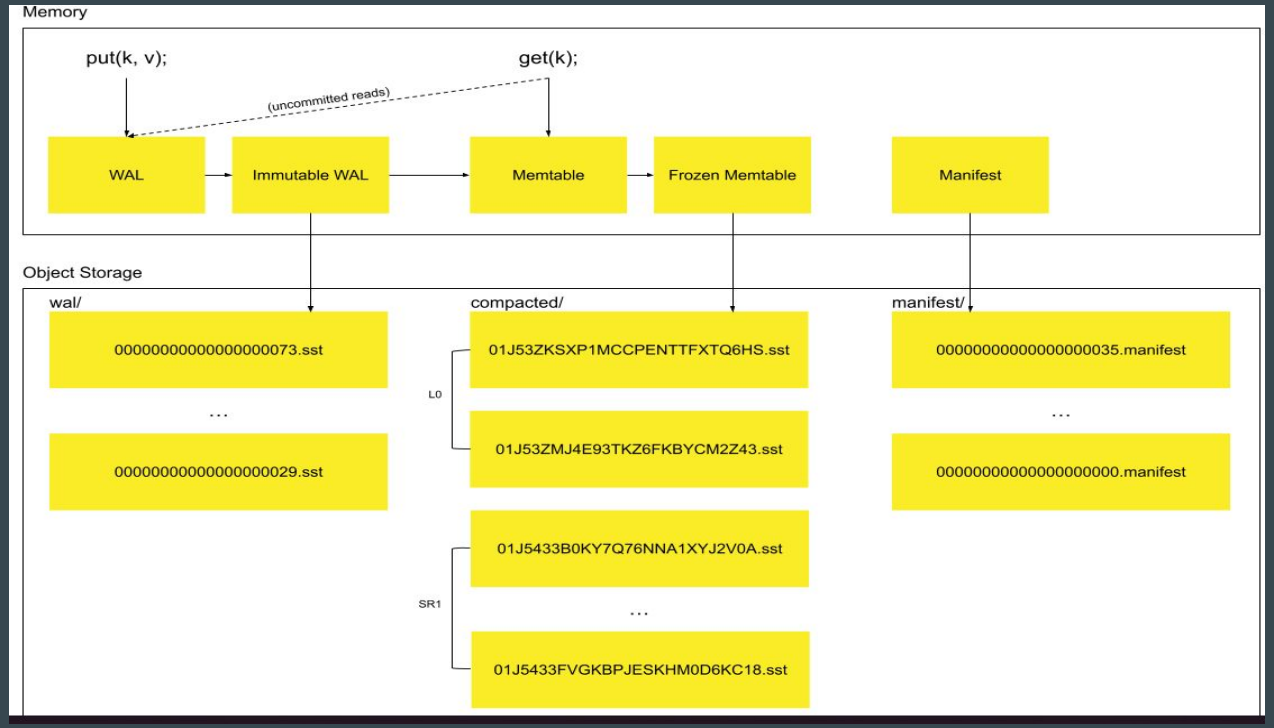
3. Compact [S2, S1] to [S3]

L0 = []
Sorted Runs = [S3]

Terminologies covered so far

- SST files: Sorted key-value pairs.
- WAL: Write Ahead Log. These are also in SST format.
- Sorted Run (SR): Set of range partitioned SST files with non-overlapping keys.
- Manifest file:
  - Describes the state of the DB.
  - i.e. Contains the list of SST files that are part of the DB.

SlateDB - Object store & memory state



- Each **manifest** describes the state of db at a point in time.
- **WAL**: Incrementally numbers, used for fencing old writer.
- All files are immutable.

SlateDB Initialization

```
wal/
  - 0004.SST
  - 0005.SST
  - 0006.SST
compacted/
  - abc.SST
  - s1-1.SST
  - s1-2.SST
manifest/
  - 0003.manifest
  - 0004.manifest
```

Object store state

1. List and find latest manifest 0004.manifest

```
{
    // 0004.manifest
    "manifest_id": 4,
    "writer_epoch": 1,
    "wal_id_last_compacted": 4,
    "l0": ["abc.SST's SsTableInfo"],
    "compacted": [
        ["s1-1.sst's SsTableInfo", "s1-2.sst's SsTableInfo"]
    ]
}
```

Current manifest - 0004.manifest

2. Read 0004.manifest and load DbState to memory

3. Create empty wal/0007.SST
4. Read and load wal/0005.SST, 0006.SST
5. Create manifest/0005.manifest

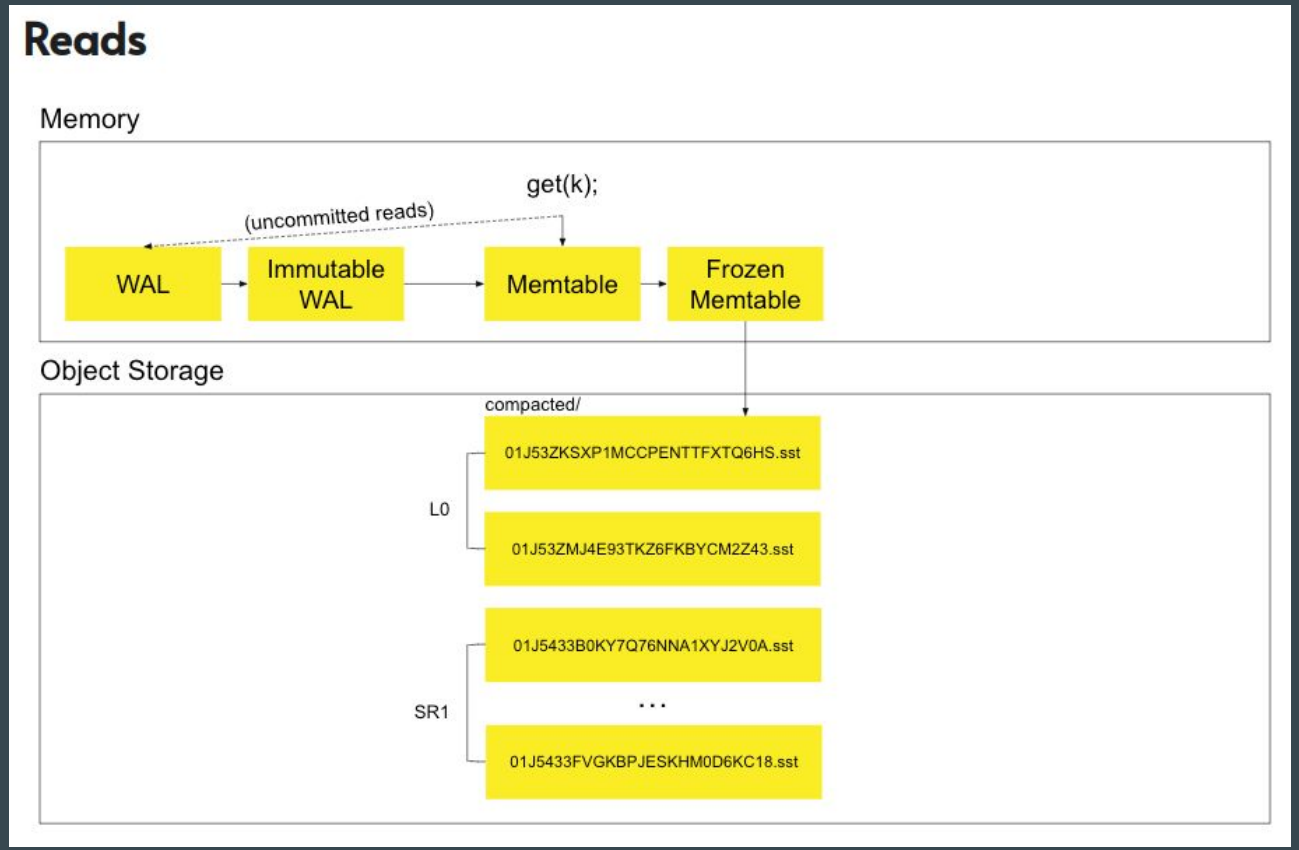
- (key, value) from 0005.SST, 0006.SST

- FirstKey for
  - abc.SST
  - s1-1.SST
  - s1-2.SST

Memory state after initialization

Initialization reads the recent manifest and loads DbState into memory.

Formal validation with https://fizzbee.io/



- Reads go from WAL -> ..-> L0 -> Sorted Runs
- In memory block caches & local disk caches are used to speed up reads.

- L0 - overlapping keys.
- SR - non overlapping keys.

- Reads go level by level.
- Metadata such as bloom filter, key range for SST files is cached in memory.
- File parts for sorted runs are cached in local disk.

Simplified Manifest

```
// Manifest to persist state of DB.
table ManifestV1 {
    // Id of manifest. Manifest file name will be derived from this id.
    manifest_id: ulong;

    // The current writer's epoch.
    writer_epoch: ulong;

    // The most recent SST in the WAL that's been compacted.
    wal_id_last_compacted: ulong;

    // The last compacted l0
    l0_last_compacted: CompactedSstId;

    // A list of the L0 SSTs that are valid to read in the `compacted` folder.
    l0: [CompactedSsTable] (required);

    // A list of the sorted runs that are valid to read in the `compacted` folder.
    compacted: [SortedRun] (required);
}

table SortedRun {
    id: uint32;
    ssts: [CompactedSsTable] (required);
}

table CompactedSsTable {
    id: CompactedSstId (required);
    info: SsTableInfo (required);
}

// Has metadata about a SST file.
table SsTableInfo {
    // First key in the SST file.
    first_key: [ubyte];

    // Offset of the index block.
    index_offset: ulong;

    // Offset of the bloom filter.
    filter_offset: ulong;
}
```

- Manifest contains current state of the DB.
- New manifest is created when
  - New writer starts.
  - L0 list updated.
  - Compaction updates [SortedRun]
  - And few other cases.
- Writer Initialization
  - Write an empty WAL.
  - Write a new manifest.

Maintenance operations

Garbage Collection

- Old Manifests and SST files not referenced by any manifests are periodically deleted.
- Manifests can be added to “checkpoints” to prevent deletion.

Compaction : Regular LSM compaction.

Maintenance operations can run in a separate process.

Checkpoints & Time travel reads

```
// Manifest to persist state of DB.
table ManifestV1 {
    // Id of manifest. Manifest file name will be derived from this id.
    manifest_id: ulong;

    // The most recent SST in the WAL that's been compacted.
    wal_id_last_compacted: ulong;

    // The most recent SST in the WAL at the time manifest was updated.
    wal_id_last_seen: ulong;

    // The last compacted l0
    l0_last_compacted: CompactedSstId;

    // A list of the L0 SSTs that are valid to read in the `compacted` folder.
    l0: [CompactedSsTable] (required);

    // A list of the sorted runs that are valid to read in the `compacted` folder.
    compacted: [SortedRun] (required);

    // A list of checkpoints that are currently open.
    checkpoints: [Checkpoint] (required);
}


// Checkpoint reference to be included in manifest to record active checkpoints.
table Checkpoint {
    // Id that must be unique across all open checkpoints.
    id: Uuid (required);

    // The manifest ID that this checkpoint is using as its `DbState`.
    manifest_id: ulong;

    checkpoint_expire_time_s: uint;
}
```

- **Why**: Keep an old DB state around longer, to allow snapshot reads.
- Checkpoint adds a reference to the manifest id in the current manifest.
- Prevents SST files in checkpointed manifests from being deleted.

- Older snapshot: Open older manifest.
- Read-only clients tail new manifest creation.

Writeable clones

```
table DbParent {
    // Path to the parent database from which this database was cloned
    path: string (required);

    // Parent checkpoint ID that this clone was derived fromP
    checkpoint: Uuid (required);
}

// Manifest to persist state of DB.
table ManifestV1 {
    // Id of manifest. Manifest file name will be derived from this id.
    manifest_id: ulong;

    // Optional details about the parent checkpoint for the database
    parent: DbParent;

    // Flag to indicate whether initialization has finished. When creating the initial manifest for
    // a root db (one that is not a clone)
    initialized: bool;

    // A list of the L0 SSTs that are valid to read in the `compacted` folder.
    l0: [CompactedSsTable] (required);

    // A list of the sorted runs that are valid to read in the `compacted` folder.
    compacted: [SortedRun] (required);
}
```

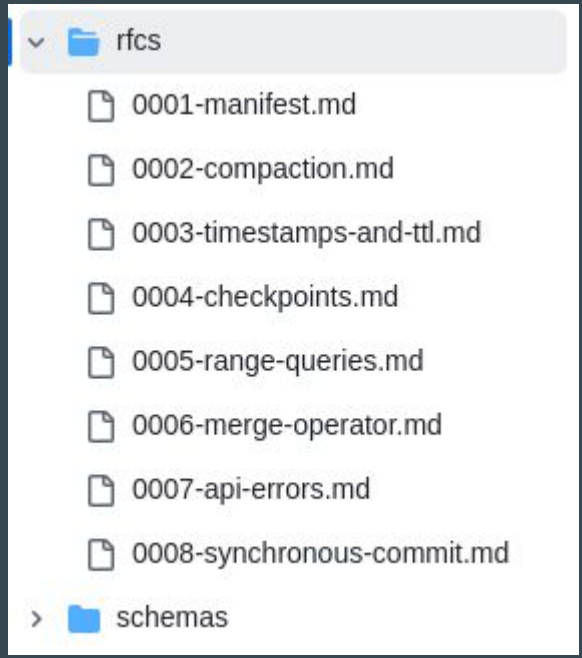
- Why: Fork application state quickly.
- Object store based design makes writeable clone very cheap!
- Reference parent DB's SST files for initialization.

Features

Feature	Application
Zero-disk architecture	Enables storage & compute disaggregation.
Single-Writer Multiple-Readers	● Scale reads independently. ● No additional mechanism needed for fencing stale writers.
Read Caching	Cache hot blocks, and improve read performance.
Remote maintenance operations	Avoids overloading serving layer.
Snapshot reads	Enables “as of” reads.
Writeable clones	Enables forking application state with negligible cost.

Want more details?

https://github.com/slatedb/slatedb/tree/main/rfcs

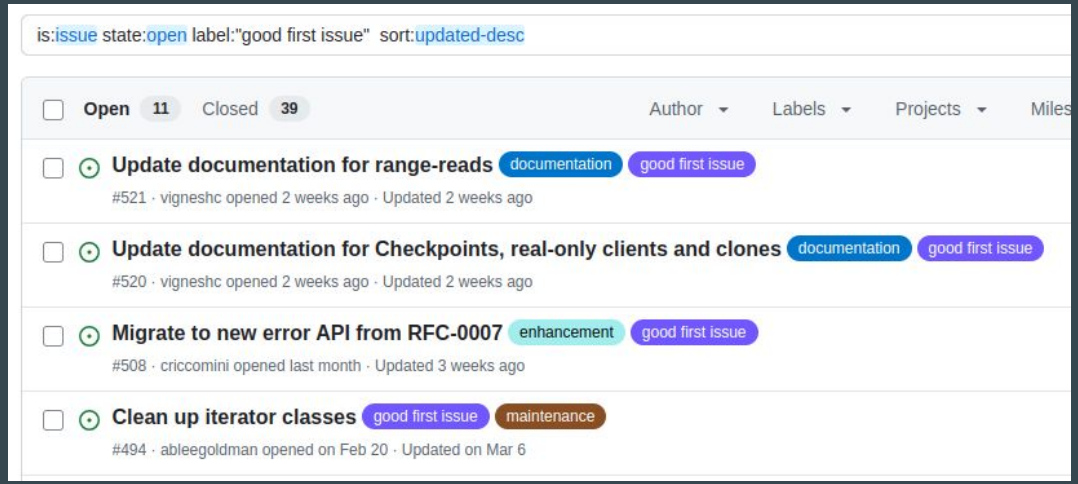


Few items on the roadmap

Feature	Application
Language bindings	Leverage SlateDB in other languages.
Change Data Capture	Replicate SlateDB to another data system, say to iceberg
Transactions	Enables stronger application semantics.

Getting Started!

- Website: https://slatedb.io/
- Demo Code: https://tinyurl.com/slatedb-demo
- Github: https://github.com/slatedb/slatedb



Summary

- Building a performant, embedded key-value store on object store is very much possible!
- With a handful of techniques, that are applicable more broadly.
- Tools like SlateDB unlocks new possibilities, and attractive cost profiles.